The Venue component uses a new hook called useEffect. The useEffect hook allows a React component to trigger behavior when the component renders and to allow that behavior to have a side effect.

In general, React tries to be a functional representation of the world, which means a React component wants to convert state directly into DOM elements without having any other effect on the system. However, we often need to do things that are not purely functional. We might need to log output. We might need to send events to other items or register to receive events from other items. In our case we want to retrieve the seating data from the server, which is updating our state based on an external source. Collectively, the things you might do that aren't directly converting state to DOM elements are called *side effects*.

React very much wants you to avoid side effects in the part of the code that renders output and provides the useEffect hook as a place to have non-functional interactions with the rest of the world. A component can have multiple useEffect hooks, and in fact, the React core teams' recommendation is to have many small effect hooks rather than one large one.

At its simplest, the useEffect hook takes just one argument. This argument is a function that would generally perform a side effect. This function is, by default, invoked after rendering, every time the component renders.

If you look at the useEffect method in the Venue component, you'll see that it's a little more involved than a simple function—there's actually an internal function called fetchData that is declared as async. Inside the fetchData function, we call our Rails server, await the response, and then use the setVenueData function from our useState hook to update the state of the component. The body of the useEffect argument calls that internal fetchData function and embeds it in a setInterval call so that it will be reinvoked every minute.

The reason for this rigmarole around the internal method is that you are not allowed to declare the argument to useEffect to be async in its own right. (To be clear, TypeScript won't let you make the argument async; you can get away with